

Software Design Document
Final Year Project Management System
(IUB – FYP-MS)



Submitted by

Umair Fareed (s23binft1m01025)

BS Information Technology | 7th Semester | Section 1M

Submitted to

Mr. Omer Riaz

Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur

Summary

The Final Year Project Management System (FYP-MS) is a web-based platform developed for the Department of Information Technology at The Islamia University of Bahawalpur to digitise and streamline the end-to-end supervision of final-year projects. It replaces the manual, email-and-paper based process currently used for collecting project proposals, SRS reports and SDD reports, and for returning supervisor evaluations to students.

This Software Design Document (SDD) translates the requirements captured in the Software Requirements Specification (SRS) into a concrete technical design. It describes how the system is structured and how its parts interact, and is intended to guide the implementation phase as well as future maintenance.

Architecturally, the system follows a three-tier client–server model built on the MERN stack – MongoDB, Express.js, React.js and Node.js. The presentation tier is a single-page React application; the application tier is a Node.js / Express REST API organised using the Model–View–Controller (MVC) pattern; and the data tier is a MongoDB document database. Role-based access control distinguishes three actors – Student, Supervisor and Administrator / Coordinator – each with a tailored set of capabilities.

The principal design considerations are security (stateless JWT authentication and bcrypt password hashing), data integrity (server-side validation and automatic deadline enforcement that locks a submission once its due date has passed), maintainability (a clean MVC separation of concerns and consistent coding conventions) and usability (a responsive, accessible interface aligned with the university's visual identity).

The remainder of this document is organised into three sections. Section 1 presents the application architecture, including the MVC components, the supporting UML diagrams (use-case, class and sequence) and the external APIs and libraries used. Section 2 details the data-model schema with an entity-relationship diagram and a description of each entity. Section 3 describes the user-interface design philosophy, the screen inventory and the controls and input constraints of each screen.

Table of Contents

Summary.....	1
1. Application Architecture.....	3
1.1 Architectural Overview.....	3
1.2 Architectural Pattern – Model–View–Controller.....	4
1.2.1 Models.....	4
1.2.2 Views.....	4
1.2.3 Controllers.....	5
1.3 Use Case Diagram.....	6
1.4 Class Diagram.....	7
1.5 Sequence Diagrams.....	8
1.6 APIs and External Libraries.....	9
1.7 Coding Conventions.....	11
2. Data Model Schema.....	12
2.1 Entity-Relationship Diagram.....	12
2.2 Entity Descriptions.....	12
2.2.1 User.....	12
2.2.2 Project.....	13
2.2.3 Submission.....	13
2.2.4 Feedback.....	14
2.2.5 Deadline.....	14
2.2.6 Notification.....	14
2.3 Relationships Summary.....	14
3. User Interface.....	16
3.1 Design Philosophy.....	16
3.1.1 Colour Scheme.....	16
3.1.2 Typography, Layout and Responsiveness.....	16
3.2 Screen Inventory.....	16
3.3 Screen Descriptions.....	17
3.3.1 Login Screen.....	17
3.3.2 Student Dashboard & Proposal Submission.....	17
3.3.3 Supervisor Review & Feedback.....	19

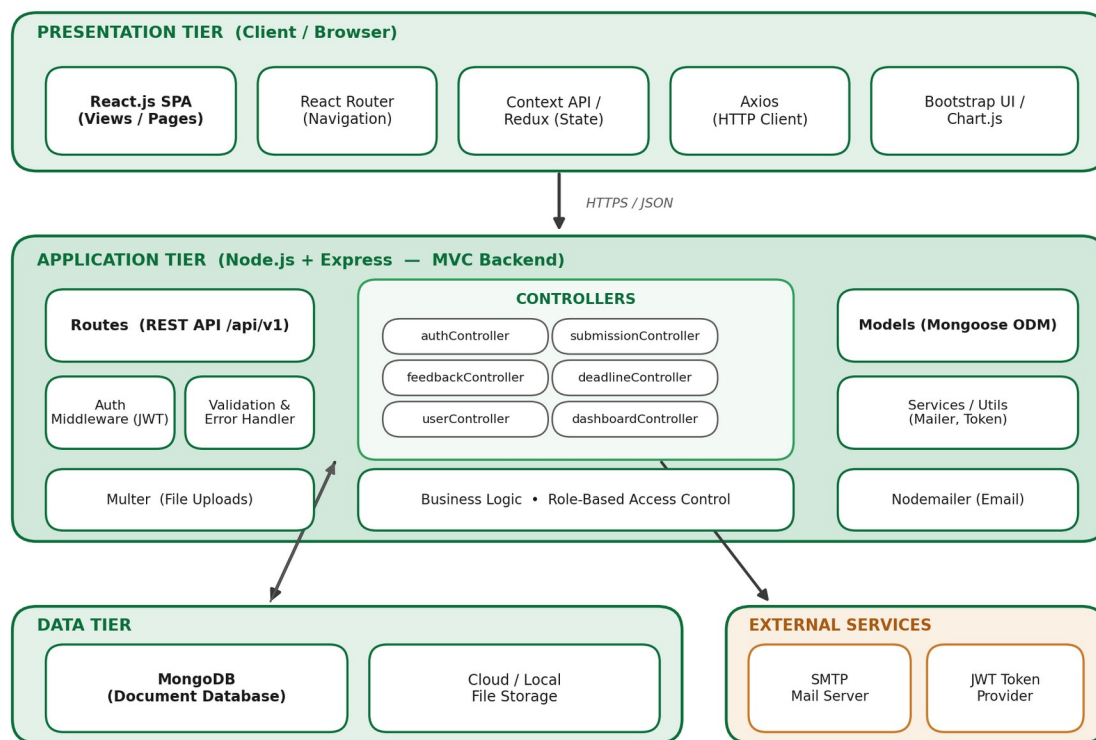
1. Application Architecture

This section describes the overall architecture of the Final Year Project Management System, the architectural pattern it follows and the components that make up the system. It also presents the supporting UML diagrams, the external APIs and libraries the application depends on, and the coding conventions adopted during development.

1.1 Architectural Overview

The application adopts a three-tier, client-server architecture in which the server-side business logic is organised according to the Model-View-Controller (MVC) pattern. Separating the system into independent tiers keeps the user interface, the application logic and the data storage loosely coupled, which improves scalability, testability and maintainability. The three tiers and the external services they rely on are shown in Figure 1.1.

Figure 1.1 – System Architecture (3-Tier MVC)



Client requests travel down the tiers; controllers orchestrate models, services and storage, then JSON responses travel back up.

Figure 1.1 – Three-tier MVC system architecture

The responsibilities of each tier are summarised below:

- **Presentation tier (client).** A single-page application built with React.js renders all screens in the browser. React Router handles client-side navigation, the Context API / Redux Toolkit manages global state (such as the authenticated user and JWT), and Axios

issues asynchronous HTTP requests to the backend. The interface is styled with React-Bootstrap, and Chart.js renders the dashboard analytics.

- **Application tier (server).** A Node.js runtime with the Express.js framework exposes a stateless REST API. Incoming requests pass through routing and middleware (authentication, validation and error handling) before reaching the controllers, which contain the business logic and coordinate the Mongoose models and helper services (mailer, token utilities). Multer handles multipart file uploads for report documents.
- **Data tier.** A MongoDB document database persists all application data, accessed exclusively through the Mongoose Object-Data-Mapping (ODM) layer. Uploaded report files are kept in cloud or local file storage, with only their references stored in the database.

Communication between the client and the server is performed over HTTPS using a stateless REST interface that exchanges JSON payloads. Every protected request carries a JSON Web Token (JWT) in the Authorization header, which the server validates on each call, so no server-side session state is required.

1.2 Architectural Pattern – Model–View–Controller

The backend is structured using the MVC pattern, which divides the application into three interconnected groups of components. In a MERN application the pattern maps onto the stack as follows: the Models are the Mongoose schemas, the Views are the React components rendered on the client, and the Controllers are the Express handler functions on the server. Because the server returns JSON rather than HTML, the View layer lives entirely in the React client while the Model and Controller layers live on the server.

1.2.1 Models

The Models define the structure of the data, the validation rules and the methods used to read from and write to the database. The proposed system uses six core models; their attributes and relationships are described in detail in Section 2 (Data Model Schema). The complete list of models is:

- User – stores account, role and profile information for students, supervisors and administrators.
- Project – links a student to a supervisor and represents the overall final-year project.
- Submission – a unified deliverable record used for the Proposal, SRS and SDD (distinguished by a type field).
- Feedback – a supervisor’s evaluation of a submission, including strengths, weaknesses, status and marks.
- Deadline – the configurable due date for each deliverable type and section.
- Notification – in-app alerts sent to users (deadline reminders and feedback notifications).

1.2.2 Views

The Views are the React pages and components that the user interacts with. The screens are listed below and described in full, together with their controls and input constraints, in Section 3 (User Interface):

- Authentication views: Login, Register, Forgot / Reset Password.

- Student views: Student Home (dashboard), Proposal Submission, SRS Report Submission, SDD Report Submission, Feedback & Marks, Deadlines.
- Supervisor views: Supervisor Dashboard, Supervisees List, Submission Review & Feedback.
- Administrator views: Admin Dashboard, Manage Users, Manage Deadlines, Assign Supervisors.
- Shared views: Profile, Notifications and a 404 / Not-Found page.

1.2.3 Controllers

The Controllers receive HTTP requests from the routes, apply the business logic, invoke the appropriate models and services, and return a JSON response with the correct status code. In line with the principle of single responsibility, the controllers are grouped by resource. The following tables list each controller and the functions it exposes.

Authentication Controller (authController)

Function	Description
register()	Creates a new user account, hashes the password and persists the user.
login()	Verifies credentials and returns a signed JWT on success.
logout()	Invalidates the client-side token / clears the auth cookie.
getProfile()	Returns the profile of the currently authenticated user.
updateProfile()	Updates the authenticated user's editable profile fields.
changePassword()	Changes the password after verifying the current one.
forgotPassword()	Generates a reset token and emails a reset link to the user.
resetPassword()	Validates the reset token and stores the new password.

Submission Controller (submissionController)

Function	Description
createSubmission()	Validates the deadline and creates a Proposal / SRS / SDD submission.
getMySubmissions()	Returns all submissions belonging to the logged-in student.
getSubmissionById()	Returns a single submission with its feedback (if any).
updateSubmission()	Updates a submission while it is still editable (before the deadline).
getSubmissionsBySupervisor()	Lists submissions of all students assigned to a supervisor.
getSubmissionsByType()	Filters submissions by deliverable type (Proposal, SRS or SDD).

Feedback Controller (feedbackController)

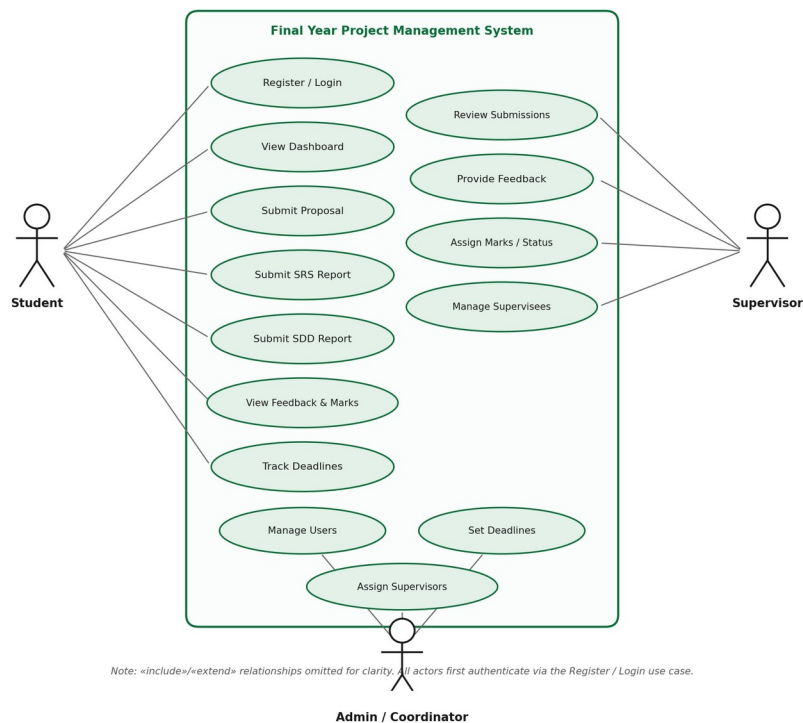
Function	Description
createFeedback()	Records a supervisor's strengths, weaknesses, status and marks for a submission.
getFeedbackBySubmission()	Retrieves the feedback associated with a given submission.
updateFeedback()	Edits previously submitted feedback or marks.

Deadline, User, Dashboard & Notification Controllers

Controller	Key functions
deadlineController	createDeadline(), getDeadlines(), getActiveDeadline(), updateDeadline(), deleteDeadline()
userController	getAllUsers(), getUserById(), createUser(), updateUser(), deleteUser(), assignSupervisor()
dashboardController	getStudentDashboard(), getSupervisorDashboard(), getAdminDashboard()
notificationController	getMyNotifications(), markAsRead(), markAllAsRead()

1.3 Use Case Diagram

The use-case diagram in Figure 1.2 captures the interactions between the three actors and the system. Every actor first authenticates through the Register / Login use case. Students submit deliverables, track deadlines and view feedback; supervisors review submissions, provide feedback and assign marks; and the administrator manages users, configures deadlines and assigns supervisors to students.

Figure 1.2 – Use Case Diagram*Figure 1.2 – Use case diagram*

1.4 Class Diagram

Figure 1.3 presents the class diagram. Each class corresponds to a Mongoose model and shows its principal attributes (with visibility and data type) and the methods that operate on it. The associations are annotated with multiplicities; for example, a User in the supervisor role supervises zero or more Projects, and each Project contains one to three Submissions, each of which may receive at most one Feedback record.

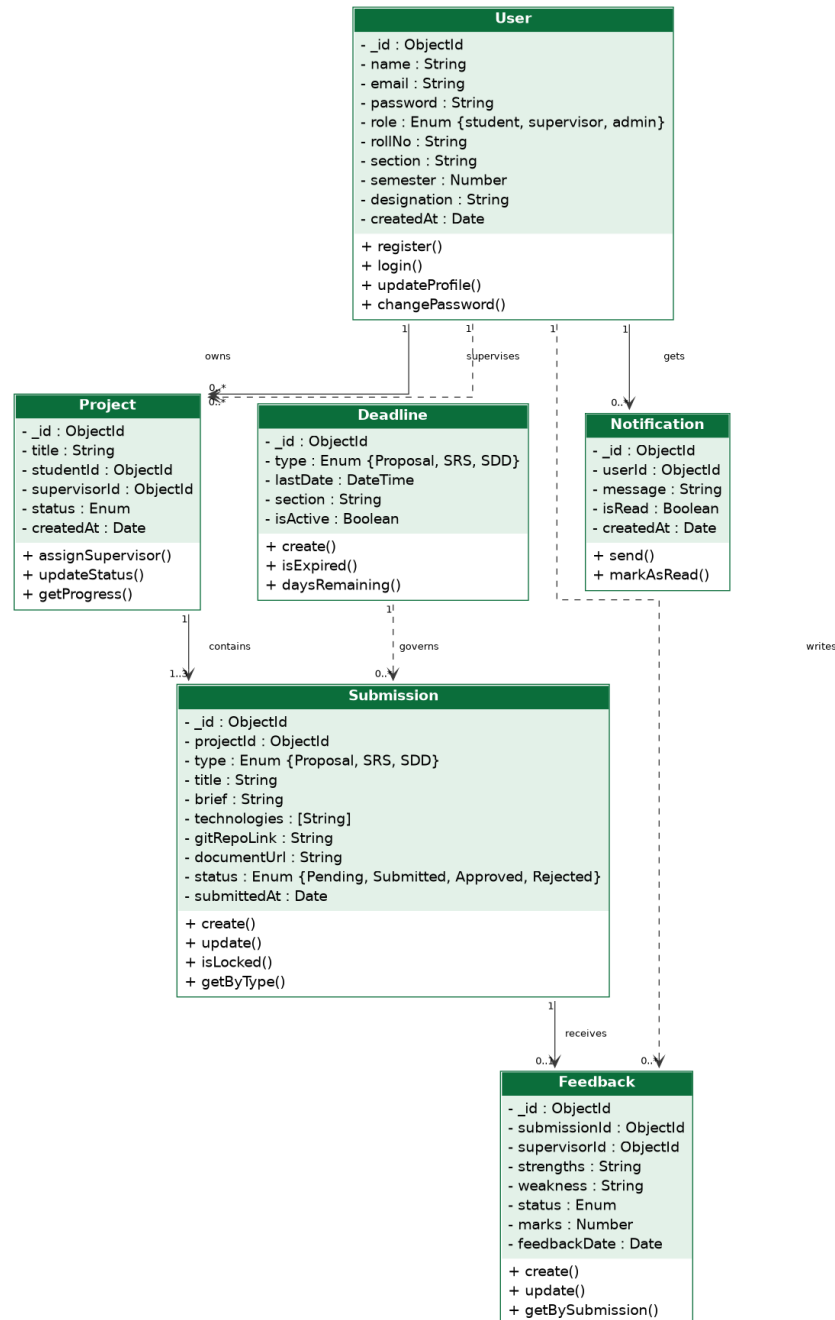


Figure 1.3 – Class diagram

1.5 Sequence Diagrams

Two representative scenarios are modelled below. Figure 1.4 shows the flow when a student submits a proposal: the request is authenticated, the active deadline is checked, and – if the deadline has not passed – the submission is persisted and a confirmation is returned. Figure 1.5 shows the supervisor evaluation flow, in which feedback and marks are stored, the submission status is updated, and the student is notified.

Figure 1.4 – Sequence Diagram: Submit Proposal

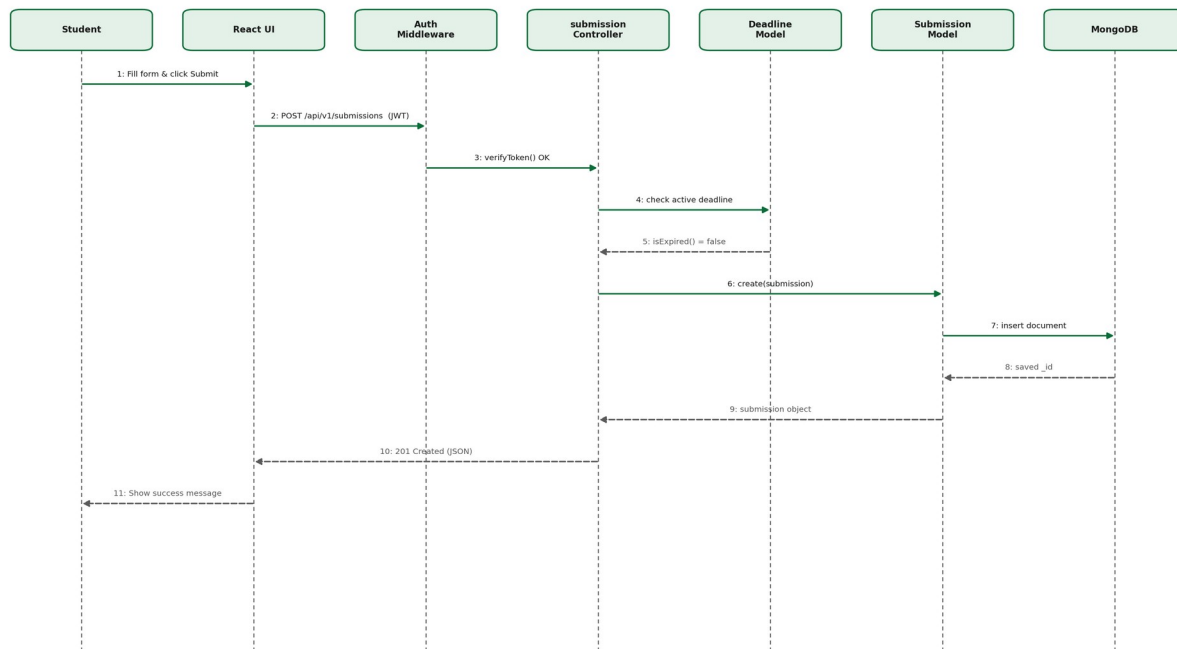


Figure 1.4 – Sequence diagram: submitting a proposal

Figure 1.5 – Sequence Diagram: Supervisor Evaluation & Feedback

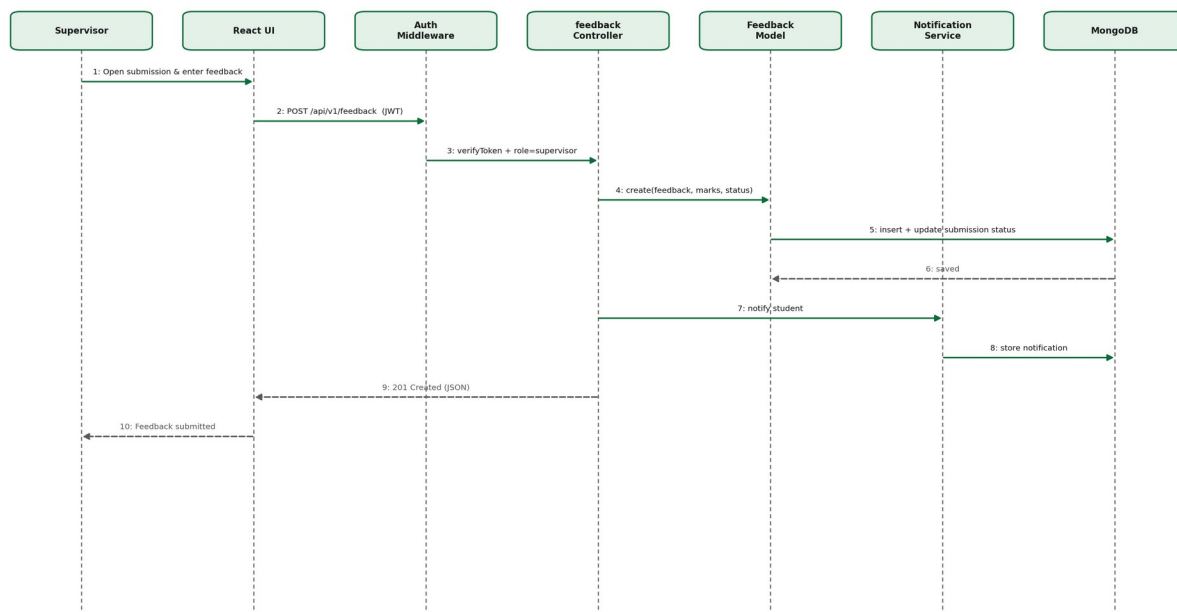


Figure 1.5 – Sequence diagram: supervisor evaluation and feedback

1.6 APIs and External Libraries

The application exposes its own RESTful API and consumes several third-party libraries and external services. All internal endpoints are versioned under the base path /api/v1 and follow

REST conventions, returning JSON together with an appropriate HTTP status code. A representative subset of the API is shown below.

Method	Endpoint	Description	Access
POST	/api/v1/auth/login	Authenticate a user and issue a JWT	Public
POST	/api/v1/submissions	Create a Proposal / SRS / SDD submission	Student
GET	/api/v1/submissions/me	List the logged-in student's submissions	Student
POST	/api/v1/feedback	Submit feedback and marks for a submission	Supervisor
GET	/api/v1/deadlines/active	Get the active deadline for a deliverable	Authenticated
POST	/api/v1/users	Create a user account	Admin

The external libraries used by the system, and the purpose of each, are listed in the following table.

Library / Service	Purpose
Express.js	Web-application framework that provides routing and middleware on the server.
Mongoose (ODM)	Models MongoDB documents as schemas and manages all database operations.
jsonwebtoken (JWT)	Issues and verifies signed tokens for stateless authentication.
bcrypt.js	Hashes and salts passwords before they are stored.
express-validator	Validates and sanitises incoming request data on the server.
Multer	Handles multipart/form-data uploads of SRS and SDD report files.
Nodemailer (SMTP)	Sends transactional emails such as deadline reminders and password resets.
Axios	Promise-based HTTP client used by the React front-end.
React Router	Declarative client-side routing between views.
Redux Toolkit / Context API	Centralised front-end state management.
React-Bootstrap	Responsive, accessible UI component library.
Chart.js	Renders charts and statistics on the dashboards.
Day.js	Lightweight date handling and the deadline countdown.

Two external services are integrated. An SMTP mail server is reached through Nodemailer to deliver notification and password-reset emails. The JWT mechanism acts as the token provider: on login the server signs a token containing the user id and role, and the client attaches this token

to every subsequent request so the server can authenticate and authorise it without storing session data.

1.7 Coding Conventions

To keep the codebase consistent and readable, the following conventions are observed throughout development:

- Naming: camelCase for variables and functions, PascalCase for React components and Mongoose models, and UPPER_SNAKE_CASE for constants and environment variables.
- Project structure: the backend is organised into models/, controllers/, routes/, middleware/, services/, config/ and utils/ folders; the front-end into components/, pages/, services/, context/ and assets/ folders.
- API design: RESTful, resource-based endpoint names under the versioned base path /api/v1, using the correct HTTP verbs and status codes.
- Language features: modern ES6+ syntax with async / await for all asynchronous operations, and centralised try / catch error handling.
- Quality tooling: ESLint and Prettier enforce a uniform style; configuration and secrets are kept out of source control in a .env file.
- Version control: Git with a feature-branch workflow and clear, descriptive commit messages.
- Documentation: meaningful inline comments and JSDoc annotations on exported functions.

2. Data Model Schema

The system stores its data in a MongoDB document database. Although MongoDB is non-relational, the application models clear relationships between collections using ObjectId references, which are depicted here in entity-relationship form for clarity. The conceptual schema comprises six entities: User, Project, Submission, Feedback, Deadline and Notification.

2.1 Entity-Relationship Diagram

Figure 2.1 shows the entity-relationship diagram. Primary keys are marked PK and foreign keys (reference fields) are marked FK. The Deadline entity is a configuration entity that is matched to submissions by deliverable type and section at the application layer rather than through a hard foreign key.

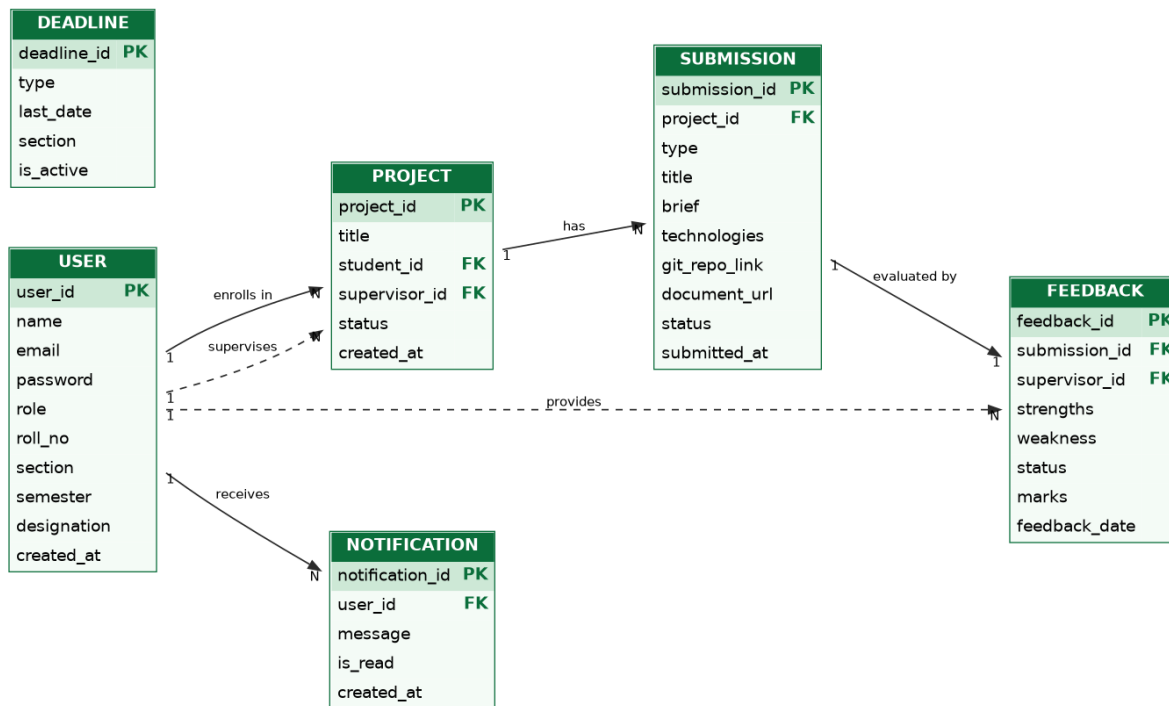


Figure 2.1 – Entity-relationship diagram

2.2 Entity Descriptions

2.2.1 User

Stores the account, role and profile information for every user of the system. The role attribute drives role-based access control and determines which views and operations the user can access.

Field	Type	Constraints	Description
user_id	ObjectId	PK, auto	Unique identifier.
name	String	Required	Full name of the user.
email	String	Required, unique	Login identifier; validated format.

Field	Type	Constraints	Description
password	String	Required, hashed	Bcrypt-hashed password.
role	Enum	student / supervisor / admin	Determines access rights.
roll_no	String	Required for students	University roll number.
section	String	Optional	Class section (e.g. 1M).
semester	Number	Optional	Current semester.
designation	String	Supervisors only	Academic designation.
created_at	Date	Auto	Account creation timestamp.

2.2.2 Project

Represents the overall final-year project and links a student to a supervisor. It acts as the parent record under which the three deliverables are grouped.

Field	Type	Constraints	Description
project_id	ObjectId	PK, auto	Unique identifier.
title	String	Required	Working title of the project.
student_id	ObjectId	FK → User	The student who owns the project.
supervisor_id	ObjectId	FK → User	The assigned supervisor.
status	Enum	active / completed	Overall project status.
created_at	Date	Auto	Creation timestamp.

2.2.3 Submission

A unified deliverable record used for the Proposal, the SRS report and the SDD report. The type field distinguishes the three, which keeps the schema DRY while still supporting deliverable-specific fields such as the proposal brief and the Git repository link.

Field	Type	Constraints	Description
submission_id	ObjectId	PK, auto	Unique identifier.
project_id	ObjectId	FK → Project	Parent project.
type	Enum	Proposal / SRS / SDD	Deliverable type.
title	String	Required	Title of the submission.
brief	String	Optional	Project brief / description.
technologies	Array	Optional	Proposed technologies.
git_repo_link	String	URL	Repository link.
document_url	String	Optional	Uploaded report file reference.
status	Enum	Pending / Submitted / Approved / Rejected	Submission status.
submitted_at	Date	Auto	Submission timestamp.

2.2.4 Feedback

Holds a supervisor's evaluation of a single submission. There is a one-to-one relationship between a submission and its feedback record.

Field	Type	Constraints	Description
feedback_id	ObjectId	PK, auto	Unique identifier.
submission_id	ObjectId	FK → Submission	The evaluated submission.
supervisor_id	ObjectId	FK → User	The evaluating supervisor.
strengths	String	Optional	Noted strengths.
weakness	String	Optional	Noted weaknesses.
status	Enum	Approved / Needs revision / Rejected	Evaluation outcome.
marks	Number	0–100	Awarded marks.
feedback_date	Date	Auto	Date feedback was given.

2.2.5 Deadline

A configuration entity that defines the due date for each deliverable type and section. It drives the countdown shown to students and the server-side lock that prevents late submissions.

Field	Type	Constraints	Description
deadline_id	ObjectId	PK, auto	Unique identifier.
type	Enum	Proposal / SRS / SDD	Deliverable the deadline applies to.
last_date	DateTime	Required	Due date and time.
section	String	Optional	Section the deadline targets.
is_active	Boolean	Default true	Whether the deadline is in effect.

2.2.6 Notification

Represents an in-app alert delivered to a user, such as a deadline reminder or a notice that feedback has been received.

Field	Type	Constraints	Description
notification_id	ObjectId	PK, auto	Unique identifier.
user_id	ObjectId	FK → User	Recipient of the notification.
message	String	Required	Notification text.
is_read	Boolean	Default false	Read / unread flag.
created_at	Date	Auto	Creation timestamp.

2.3 Relationships Summary

The relationships between the entities, with their cardinalities, are as follows:

- A User (student) owns one Project, and a User (supervisor) supervises many Projects – one-to-many.
- A Project contains one to three Submissions (Proposal, SRS, SDD) – one-to-many.
- A Submission receives at most one Feedback record – one-to-one.
- A User (supervisor) provides many Feedback records – one-to-many.
- A User receives many Notifications – one-to-many.
- A Deadline governs many Submissions of a given type and section – enforced at the application layer.

3. User Interface

This section describes the design philosophy of the user interface, lists the screens that make up the application, and explains the data, controls and input constraints of the principal screens.

3.1 Design Philosophy

The interface follows a clean, minimal, card-based academic dashboard style. The visual language is built around the university's identity and prioritises clarity, consistency and quick scanning of status information (deadlines, submission status and marks).

3.1.1 Colour Scheme

The palette is anchored by the IUB green and supported by neutral greys and white, with semantic colours reserved for status and alerts.

Role	Colour	Hex	Usage
Primary	IUB Green	#0B6E3B	Top bar, primary buttons, headings, active navigation.
Surface	White / Light grey	#FFFFFF / #F4F7F5	Page background and cards.
Text	Ink	#1A1A1A	Body text.
Warning	Amber	#C77B2B	Deadline banners and countdown.
Success	Green	#0B6E3B	Submitted / approved status.
Danger	Red	#C0392B	Errors and rejected status.

3.1.2 Typography, Layout and Responsiveness

- **Typography:** a sans-serif family (Segoe UI / Arial / system stack) is used throughout, with a clear hierarchy of heading, sub-heading and body sizes for legibility.
- **Layout:** a persistent top bar and a left navigation sidebar frame a card-based content area; forms use a single-column layout with generous spacing.
- **Responsiveness:** built on the responsive Bootstrap grid. The primary target is desktop and laptop screens, with tablet and mobile supported – the sidebar collapses into a menu on narrow viewports.
- **Accessibility:** sufficient colour contrast, labelled form controls, focus states and full keyboard navigation.

3.2 Screen Inventory

The application is composed of the following screens, grouped by the role that uses them:

Group	Screens
Authentication	Login, Register, Forgot / Reset Password
Student	Student Home, Proposal Submission, SRS Report, SDD Report, Feedback & Marks, Deadlines
Supervisor	Supervisor Dashboard, Supervisees List, Submission Review & Feedback

Group	Screens
Administrator	Admin Dashboard, Manage Users, Manage Deadlines, Assign Supervisors
Shared	Profile, Notifications, Not-Found (404)

3.3 Screen Descriptions

3.3.1 Login Screen

The entry point to the application. It presents a centred card with the system identity and the credential fields, and provides links for password recovery and registration. The screen is shown in Figure 3.1.

Figure 3.1 – Login / Authentication Screen

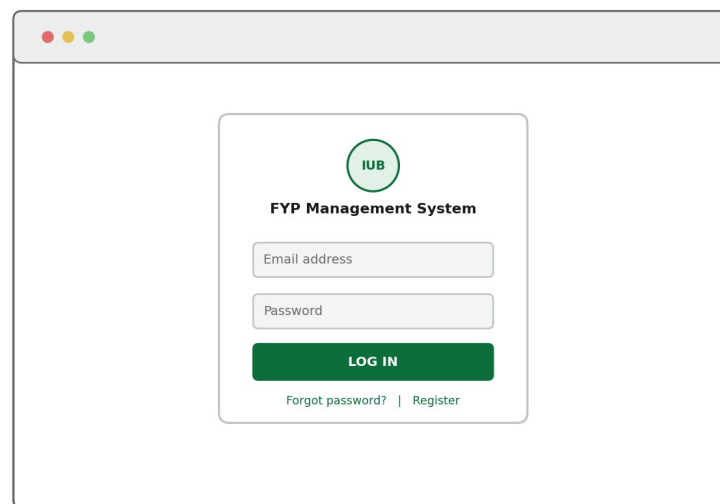


Figure 3.1 – Login / authentication screen

Control	Type	Description	Input constraint
Email address	Text field	User's login email	Required; valid email format.
Password	Password field	Account password	Required; minimum 8 characters.
Log in	Button	Submits the credentials	Enabled when both fields are filled.
Forgot password / Register	Links	Navigate to recovery / sign-up	–

3.3.2 Student Dashboard & Proposal Submission

After logging in, a student lands on the dashboard. The left sidebar provides navigation between the deliverables, and the content area shows the selected deliverable form – here, the Proposal Submission. A banner displays the due date and a live countdown, and a status chip reflects the

current state of the submission. Once the deadline passes, the form is locked and the controls are disabled. The screen is shown in Figure 3.2.

Figure 3.2 – Student Dashboard & Proposal Submission

Figure 3.2 – Student dashboard and proposal submission

Control	Type	Description	Input constraint
Sidebar navigation	Links	Switch between Home, Proposal, SRS, SDD	–
Deadline banner	Read-only	Shows last date and days remaining	Auto-calculated.
Proposal Title	Text field	Title of the project	Required; max 150 characters.
Project Brief	Text area	Short description of the project	Required; 50–1000 characters.
Technologies	Text field	Proposed technology stack	Required.
Git Repository Link	Text field	Link to the code repository	Optional; valid URL format.
Submit	Button	Submits the proposal	Disabled after the deadline.
Cancel	Button	Clears / discards the form	–
Status	Chip / badge	Current submission status	System-set.

3.3.3 Supervisor Review & Feedback

Supervisors open a student's submission to review it and record an evaluation. The left panel shows the read-only submission detail; the right panel is the feedback form for the strengths, weaknesses, status and marks. Submitting the form stores the evaluation, updates the submission status and notifies the student. The screen is shown in Figure 3.3.

Figure 3.3 — Supervisor Review & Feedback Screen

Figure 3.3 – Supervisor review and feedback screen

Control	Type	Description	Input constraint
Submission detail	Read-only panel	Student, type, title, brief, technologies, repo	Non-editable.
Strengths	Text area	Positive points of the submission	Required.
Weaknesses	Text area	Areas needing improvement	Required.
Status	Dropdown	Approved / Needs revision / Rejected	Required; one option.
Marks	Number field	Awarded marks	Required; integer 0–100.
Submit feedback	Button	Saves feedback and notifies the student	Enabled when required fields are valid.

— End of Document —